

Optical Pulse Sensor (KY-039)

Hardware Overview

What is it?

The **KY-039** is a very primitive optical sensor used to measure heart rate. It uses a technique called *Photoplethysmography* (PPG). It shines a light into your skin and measures how much light bounces back or passes through.

Electrical Explanation

The hardware consists of exactly two components:

- **Infrared LED:** Shines a bright, invisible light into your finger.
- **Phototransistor:** A light-sensitive receiver.

Every time your heart beats, a blood rushes into the capillaries in your finger. Because blood is opaque, it temporarily blocks some of the infrared light. The phototransistor detects this microscopic dip in light, changing its analog output voltage.

Key Hardware Facts

- **Connection:** Analog Pin (e.g. A3).
- **Voltage:** 5V
- **Signal Flow:** IR LED → Finger Capillaries → Phototransistor → Analog Voltage → ADC → Processor → *Digital Filter (Software)*.

Software & Programming

kcomp_KY039.h

Because the hardware is incredibly basic, the raw signal is filled with electrical noise and light interference. This library acts like a primitive Digital Signal Processor (DSP). It runs a filter to clean noise, tracks the room's baseline lighting, and calculates metrics like Heart Rate Variability (HRV).

Usage: `#include <kcomp_KY039.h>`

The Data Structure

When you call `updateHeartBeat()`, you receive a `heartbeat_data_t` struct containing:

Field	Type	Description
<code>beat_detected</code>	<code>bool</code>	<code>true</code> for exactly one microsecond the moment a pulse is found.
<code>bpm</code>	<code>uint8_t</code>	The estimated Beats Per Minute.
<code>hrv_ms</code>	<code>uint16_t</code>	Heart Rate Variability. The physical difference in milliseconds between your last two heartbeats. (Higher HRV usually means you are relaxed!)
<code>signal</code>	<code>uint16_t</code>	The clean, mathematically filtered waveform.

Function Reference

Function Name	Description & Parameters
<code>void initHeartbeat(uint8_t pin)</code>	Setup: Initializes the sensor. Must be called in <code>setup()</code> . ▪ <code>pin</code> : Analog Pin (e.g. A3).
<code>heartbeat_data_t updateHeartbeat()</code>	The Worker: CRITICAL! Call this in your <code>loop()</code> as often as possible. If you don't call this, the math stops working.

⚠ Pitfalls & Troubleshooting

- The `delay()` Death Trap

This sensor measures things happening in milliseconds. If you put a `delay(100)` anywhere in your main `loop()`, the processor will be "asleep" while the heartbeat happens, and the library will miss the pulse entirely.

The Fix: Write non-blocking code using `millis()`.

- The Death Grip

If you press your finger against the sensor too hard, you will physically squeeze all the blood out of the capillaries in your fingertip (blanching).

The Fix: Rest your finger *lightly* over the LED and receiver. Do not push down.



Sample Program

This program monitors the sensor at maximum speed. It only prints data to the Serial Monitor on the exact millisecond a heartbeat is detected!

```
#include <kcomp_Heartbeat.h>

// Connect the KY-039 'S' Pin to Analog 0
#define PULSE_PIN A0

void setup() {
    Serial.begin(9600);

    initHeartbeat(PULSE_PIN);

    Serial.println("K-Comp EKG System Online");
    Serial.println("Please place finger LIGHTLY on the sensor...");
}

void loop() {
    // 1. Run the DSP engine at maximum speed
    heartbeat_data_t heart = updateHeartbeat();

    // 2. Only output data when the wave hits its peak!
    if (heart.beat_detected) {
        Serial.print("❤ BEAT! | BPM: ");
        Serial.print(heart.bpm);
        Serial.print(" | HRV: ");
        Serial.print(heart.hrv_ms);
        Serial.println(" ms");
    }

    // Notice: There is absolutely NO delay() in this loop!
}
```

💡 **Try it out:** You can find the full sample program **SimpleHeartBeatDetectorK039** in the K-Comp examples folder!